

Near-Duplicate Build Failure Detection from Continuous Integration Logs

Mingchen Li
mingchen.li@helsinki.fi
University of Helsinki
Helsinki, Finland

Jesse Nyyssölä
jesse.nyyssola@helsinki.fi
University of Helsinki
Helsinki, Finland

Mika Mäntylä
mika.mantyla@helsinki.fi
University of Helsinki & University of Oulu
Helsinki & Oulu, Finland

Matti Luukkainen
matti.luukkainen@helsinki.fi
University of Helsinki
Helsinki, Finland

Abstract

Addressing build failures is important in software development, particularly within Continuous Integration and Deployment (CI/CD) pipelines. A common issue arises when a developer encounters a new build failure that has occurred in the past. We refer to such build failures as near-duplicate build failures. We collect build logs from a GitHub Actions CI/CD pipeline. We label near-duplicate build failures by identifying those that have the same failed tests. We propose a framework for detecting near-duplicate build failures through log similarity analysis. As the majority of log lines in failed builds are similar to passing logs, we propose using an Out-of-Vocabulary Detector (OOVD) filtering to identify only failure-relevant lines and to improve near-duplicate detection accuracy. Our results suggest a noticeable improvement using OOVD filtering across all metrics for Top-K results ($K = 5$, precision@K: 0.864 vs. 0.526 and MAP@K: 0.941 vs. 0.814). Finding near-duplicate build failures can be an important software engineering challenge that, to the best of our knowledge, has not been studied in the past.

CCS Concepts

• **Software and its engineering** → **Software defect analysis**; • **Information systems** → *Top-k retrieval in databases*; *Similarity measures*.

Keywords

Near-duplicate Detection, Build Failure, Log Analysis, Out-Of-Vocabulary, Continuous Integration and Deployment, Software Anomaly Detection

ACM Reference Format:

Mingchen Li, Mika Mäntylä, Jesse Nyyssölä, and Matti Luukkainen. 2025. Near-Duplicate Build Failure Detection from Continuous Integration Logs. In *21st International Conference on Predictive Models and Data Analytics in Software Engineering (PROMISE '25)*, June 26, 2025, Trondheim, Norway. ACM, New York, NY, USA, 4 pages. <https://doi.org/10.1145/3727582.3728691>



This work is licensed under a Creative Commons Attribution 4.0 International License. PROMISE '25, Trondheim, Norway

© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1594-5/2025/06
<https://doi.org/10.1145/3727582.3728691>

1 Introduction

Continuous integration and continuous delivery (CI/CD) pipelines are a very widely used software engineering practice. In CI/CD, software builds are executed continuously, often triggered by every commit to the codebase. As a result, a large number of builds are generated over time [5, 9].

Build failures are common in CI/CD pipelines. They stem from various causes, including compile errors, configuration mismatches, missing dependencies, changes in the build environment, or test executions [6]. Failed tests have been found to be the most common cause of build failures [1]. Build failures are seen to be more informative than passing builds, as they provide more actionable insight [7]. The prediction of build failures has also been studied [11, 12].

To the best of our knowledge, the detection of near-duplicate build failures has not been studied in previous research. Near-duplicate build failures refer to instances where a new build failure very closely resembles a previously encountered one. Proper techniques to recognize build duplicates may save effort, as they reduce redundant issue investigation or can at least give developers hints about similar build failures that have occurred in the past. We investigated near-duplicate builds using logs generated by CI/CD pipelines.

The concept of near-duplicate detection has previously been studied in near-duplicate document detection [4], near-duplicate video detection [3], and near-duplicate image detection [13]. Of these, the closest one to near-duplicate build detection is near-duplicate document detection, albeit still quite different.

In the context of software engineering, near-duplicate build failure detection shares similarities with duplicate bug report detection [15] and code clone detection [14]. Duplicate bug reports occur when multiple submissions describe the same underlying bug within the same software. However, unlike bug reports, which are written by humans, build failure logs are typically machine-generated and often contain large amounts of noisy and irrelevant information [5]. Code clone detection, on the other hand, focuses on identifying duplicates in the source code. Near-duplicate code detection is commonly referred to as semantic code clones defined as: "two pieces of code have different texts but the same functionality" [14].

In this paper, we propose a novel approach for identifying near-duplicate build failures by analyzing log data of a CI/CD pipeline

through textual similarity analysis. Additionally, we examine the impact of filtering input log data using the Out-of-Vocabulary Detector (OOVD) on the performance of our similarity analysis. OOVD is a model designed to identify words or phrases in a dataset that are not part of the existing vocabulary [10]. Log lines containing out-of-vocabulary (OOV) terms can be linked to failure-related patterns and behaviors, providing a method for highlighting significant deviations in failed test cases.

2 Research questions

RQ 1: *When identifying the most similar build runs for a passing or failed build, what proportion of the retrieved runs in the top results is relevant?* Identifying similar runs for passing builds serves as a control test, leveraging the tendency of passing runs to be duplicates. This assesses how well duplicate builds are detected. However, our main focus is on failed build runs, evaluating how effectively near-duplicate test failures are identified.

RQ 2: *Does OOVD filtering on log files increase the number of relevant hits compared to the original log files?* Since the logs of passing and failed build runs share many similarities, we aim to determine whether highlighting their differences through OOVD filtering improves our detection method.

3 Methods

3.1 Project

The data comes from Oodikone¹, an open source web-based analytics tool developed by the computer science department of University of Helsinki. The tool serves roughly 500 users, analyzing study registry data to provide insights into student performance. The project is a React.js single-page app with a monolithic Node.js backend. The codebase consists of approximately 50,000 lines of code, with 55 % of it written in TypeScript. The development team commits several times daily, each triggering the CI/CD pipeline, leading to staging releases upon successful builds. Manually tagged commits result in production releases, averaging three per week.

3.2 Data

Our analysis focuses only on the GitHub Action workflow of Oodikone concerning end-to-end (E2E) testing and staging deployment, which is triggered to run by repository events, such as code commits or pull requests. The workflow consists of multiple jobs, each containing one or more test suites, which are further divided into individual test cases. Jobs serve as E2E tests to verify the proper functionality of specific parts of the website under specific scenarios. For simplicity, we refer to E2E jobs as tests. All of the tests are implemented with Cypress, a JavaScript testing framework.

The log data consists of the terminal outputs from the virtual machine used in the workflow. In addition to detailing steps that define the CI/CD pipeline, the logs contain metadata, such as the runtime environment, version numbers of tools being used, and timestamps for each step. The primary data of interest consists of the Cypress reports, which display the total number of tests executed, the counts of passed, failed, and pending tests, as well as the time taken for the test suite to complete. Furthermore, for

each test, the reports also provide information on test execution status, console output, error messages, and short stack traces for failed tests. Artifacts generated by Cypress, such as screenshots and video recordings of the testing process, were not considered for analysis.

We collected 410 build logs, including 71 failed and 339 passing, with an average of 9,000 log lines per build run. The dataset spans roughly six months, from July 31, 2024, to February 10, 2025. Further past data could not be collected due to the default restriction placed on GitHub Actions log data being archived for only 90 days.

The four tests with the highest frequency of failed logs were selected for analysis, specifically as the target cases for identifying the most similar failed build runs. These tests had 28, 16, 12, and 11 failed instances. Tests with fewer failed instances were excluded as target runs due to insufficient data but were included in the pool of candidates for similarity retrieval.

3.3 Similarity metrics

In order to determine the similarity between two runs, we employ the following text distance measures on build logs: Jaccard distance, cosine distance (cosine similarity), containment distance (the degree of the smaller dataset being a subset of the larger one) and compression distance (implemented with the bzip2 compression algorithm). To quantify the overall similarity between the build failure logs, these individual distance metrics are aggregated into a single value. First, the Z-score is computed for each metric to standardize the values relative to the sample mean and standard deviation. Next, the Z-scores are summed to obtain a composite similarity score, Z-score sum, which represents the combined similarity across all four metrics. Ultimately, only the Z-score sum is used to evaluate similarity. LogDelta² was used to compute the metrics from logs.

3.4 Performance metrics

Our approach incorporates metrics commonly used for evaluating recommender systems and other near-duplicate detection tasks [2, 3]. For all performance metrics, the evaluation is restricted to the Top- K and Top- R results. For our analysis, we selected $K = 5$ for two key reasons. First, developers for a project of this size are unlikely to look beyond five recommendations for a given build failure. Second, a larger number of retrieved documents would approach the total number of failing job instances, rendering the Top- K suggestions redundant from a developer's perspective. Top- R analysis provides insights on the rank order of all relevant items. R varies, as it is the total number of relevant items per failed test.

We employ precision as a metric to evaluate the effectiveness of retrieving relevant items. To assess the effectiveness of retrieving relevant items in a ranked order, we use mean average precision (MAP), where identifying relevant items in lower ranks results in a reduced score. For Top- K analysis, the performance metrics include precision@ K and MAP@ K , while for Top- R analysis, we use precision@ R (same as R-precision), MAP@ R , and MAP@Full. The key distinction between MAP@ K , MAP@ R , and MAP@Full is that MAP@ K and MAP@ R consider the relevant items only within the Top- K and Top- R results, respectively, whereas MAP@Full

¹<https://github.com/UniversityOfHelsinkiCS/oodikone>

²<https://github.com/EvoTestOps/LogDelta>

evaluates the relevant items across the entire dataset. MAP@Full provides insight into the ranking of all relevant items without the constraints of a Top- R limitation. For each metric, the mean was calculated across each of the four failed tests. Subsequently, the overall average was calculated as the average of these four individual means. For passing builds, the mean of each metric is computed directly across all runs.

3.5 Labeling

In this paper, we define near-duplicate builds as those in which the same test fails across multiple executions or all tests pass successfully. Under these conditions, we consider the builds to be nearly identical despite minor variations. However, it is important to note that the criteria for near-duplicate builds may vary across studies, depending on the specific goals and constraints of the analysis.

A build run is considered passing if all tests within the workflow are successful. It is deemed failed if any test fails, with the build being labeled based on the names of the failed tests, which may include one or more.

GitHub API directly provides details on each build run, including its unique run ID, run status, and failed tests, allowing logs to be automatically labeled. The calculation of our performance metrics requires labeling a list of relevant items to determine whether the retrieved runs are hits or misses. We determine relevant items through set overlap matching, where relevance is defined exclusively by the intersection between the label sets of the target run and the retrieved run. Therefore, any partial matching of labels constitutes a relevant hit: a retrieved run is a hit if and only if it shares at least one failed test with the target run. Passing build runs are not differentiated by tests in their labeling, as the dataset contains only one type of passing build.

3.6 OOVD filtering

The Out-of-Vocabulary Detector (OOVD) [10] is used to identify OOV log lines in failed runs by establishing a baseline vocabulary from normal passing runs. The OOVD model is first trained on log data exclusively from passing runs, allowing the typical distribution of words and phrases present in successful executions to be learned. The content format is tokenized into words and the vectorization based on word count. Once trained, OOVD is applied to failed runs to detect lines containing OOV terms. To clarify, passing runs are not subjected to OOVD filtering because, by definition, the process would not change the log files. Data exploration revealed that log files from passing runs could have log line OOVD scores (the number of OOVD tokens) of up to one due to normal variability. To capture only important OOV lines, the OOVD filtering threshold is set above one, resulting in filtered log files containing only lines with more than one OOV token. The OOVD implementation used is available in the LogLead³ tool [8].

3.7 Data processing pipeline

GitHub Actions log files were collected via GitHub REST API, cleaned by removing unnecessary system files and duplicate subfolders, followed by labeling with dates, unique run IDs, run status, and failed tests. The log analysis was conducted without a parser.

³<https://github.com/EvoTestOps/LogLead>

The logs were preprocessed using regex masking before OOVD filtering and textual similarity analysis.

Regex masking was applied to all log lines to remove dynamic elements, such as timestamps and process IDs, replacing them with standardized placeholders. For OOVD, vectorization was based on word count, while for similarity analysis, TF-IDF was used. Failed logs were selected based on the four most frequently failed tests. Passing logs were used to establish a baseline for OOVD filtering. Failed logs were then separated into two datasets: with OOVD filtering and without. Textual similarity was computed pairwise between the target run and each comparison run, then ranked by Z-score sum, and the Top- K or Top- R results were selected. Performance metrics were calculated based on hits.

4 Results

Our initial analysis related to passing runs (RQ 1), is summarized in the left half of Table 1. It serves as a control test in which passing runs are individually compared against the full dataset containing both passing and failed runs. Both the Top- K and Top- R results demonstrate a strong tendency to retrieve duplicate passing runs, with OOVD filtering enhancing both the retrieval rate and the rank order compared to the baseline. As indicated by the identical metric values of precision@ R , MAP@ R and MAP@Full, with OOVD filtering (1.000), the rank order remained unchanged in all cases of target comparisons involving passing runs, with all relevant items in the Top- R ranked at the very top.

The results regarding the failed tests (RQ 1) are summarized in the right half of Table 1. The results average over $n = 28 + 16 + 12 + 11$ failed test instances by first computing the mean of each metric within the four failed test groups, and then aggregating these means. The results show that OOVD Top- K MAP@ K is higher than precision@ K , indicating that relevant items are consistently ranked at the very top of the OOVD Top- K results (0.941 vs. 0.864). This trend is especially evident in the baseline results, where MAP@ K (0.814) far exceeds precision@ K (0.526). These findings are also consistent for MAP@ R and precision@ R (0.712 vs. 0.378).

OOVD filtering (RQ 2) results are in the bottom row of Table 1. It significantly improves the inclusion of relevant items in the Top- K results in terms of precision@ K (0.864 vs. 0.526), also seen in the Top- R results (0.563 vs. 0.378). The retrieval of all relevant items reflects a moderate overall retrieval rate (OOVD: precision@ R = 0.563, Baseline: precision@ R = 0.378). Despite a high ranking among the Top- R results demonstrated by MAP@ R , MAP@Full shows that there are a significant number of relevant items of lower ranking outside of the Top- R results with and without OOVD filtering.

5 Limitations

The results concerning precision exhibit a bias towards tests with a higher frequency of failures. This is observed as the number of relevant hits identified is consistently higher for tests with a greater number of failed instances within the dataset. This suggests that the effectiveness of our approach in identifying relevant items increases with the prevalence of failing tests, indicating a dependency on the distribution of failures. However, it is important to emphasize that, in practice, failure occurrences are unlikely to be uniformly

Table 1: Performance Metrics for tested scenarios ($K = 5$)

	Passing ($n = 339$)					Failing ($n = 28 + 16 + 12 + 11$)				
	precision@K	MAP@K	prec@R	MAP@R	MAP@Full	precision@K	MAP@K	prec@R	MAP@R	MAP@Full
Baseline	0.932	0.901	0.900	0.965	0.864	0.526	0.814	0.378	0.712	0.387
OOVD	1.000	1.000	1.000	1.000	1.000	0.864	0.941	0.563	0.874	0.489

distributed, as is the case with our data coming from a real-world open-source project.

Since this study focuses on logs from a single open-source project, our findings may not generalize to projects with different architectures or CI/CD configurations. Furthermore, newly introduced substantial changes to the testing scheme can make the use of OOVD filtering problematic, as such changes might introduce terms that are deemed as OOV despite not being related to any failure patterns. Nevertheless, the results remain informative in stable projects or during periods when the testing scheme remains consistent.

In log analysis, metrics, such as recall and F-score are widely used but problematic when K is small relative to the number of relevant items, as they result in artificially low values that improve with larger K , misleadingly suggesting better performance. For Top- K , we exclude recall and F-score due to the high number of relevant items; for Top- R , we exclude recall@ R and F-score@ R , as they are redundant with precision@ R by definition. The most commonly used metric for evaluating the performance of Top- K duplicate bug report detection methods is recall@ K , denoting the average of duplicates found within the Top 1, 2, ..., K results [2]. However, in our case, this approach is not suitable, as the number of relevant items is much higher than in duplicate bug report detection, where typically only one item is being considered relevant. Similar to recall, recall@ K would have been misleading in our case.

6 Conclusions

In this paper, we introduced the problem of near-duplicate build detection and highlighted similar tasks studied in the past, such as near-duplicate document detection [4] and duplicate bug report detection [15]. Near-duplicate build failure detection can potentially reduce the effort required for failure resolution by identifying candidate near-duplicate failures from past build runs.

We collected data from a software project with 410 builds, each averaging 9,000 log lines. Several measures were used to demonstrate that applying OOVD filtering to log lines improves the accuracy of duplicate build detection compared to using all log lines. We think this improvement is due to the fact that the log contents of all builds share many redundant similarities, which the OOVD filter then removes.

This study serves as an initial exploration of the topic. Future work could expand by incorporating build data from multiple projects and exploring alternative filtering or weighting techniques beyond OOVD. The usefulness of linking a current failure to a similar past failure should be empirically assessed in collaboration with software developers. Conceptually, retrieving information on past similar build failures seems beneficial for resolving current failures, but further validation and better tooling are required.

Data availability and Acknowledgment

Details on the implementation and the full dataset are available in the Zenodo replication package at this link: [10.5281/zenodo.15080379](https://doi.org/10.5281/zenodo.15080379).

This work is funded by the Research Council of Finland (Decision No. 359861) under the academic project MuFAno. The authors acknowledge the CSC-IT Center for Science, Finland, for providing computational resources.

References

- [1] Moritz Beller, Georgios Gousios, and Andy Zaidman. 2017. Oops, my tests broke the build: An explorative analysis of travis ci with github. In *2017 IEEE/ACM 14th International conference on mining software repositories (MSR)*. IEEE, 356–367.
- [2] Markus Borg, Per Runeson, Jens Johansson, and Mika V Mäntylä. 2014. A replicated study on duplicate detection: Using Apache Lucene to search among Android defects. In *Proceedings of the 8th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement*. 1–4.
- [3] Chien-Li Chou, Hua-Tsung Chen, and Suh-Yin Lee. 2015. Pattern-Based Near-Duplicate Video Retrieval and Localization on Web-Scale Videos. *IEEE Transactions on Multimedia* 17, 3 (2015), 382–395. doi:10.1109/TMM.2015.2391674
- [4] Hannaneh Hajishirzi, Wen-tau Yih, and Aleksander Kolcz. 2010. Adaptive near-duplicate detection via similarity learning. In *Proceedings of the 33rd international ACM SIGIR conference on Research and development in information retrieval*. 419–426.
- [5] Michael Hilton, Timothy Tunnell, Kai Huang, Darko Marinov, and Danny Dig. 2016. Usage, costs, and benefits of continuous integration in open-source projects. In *Proceedings of the 31st IEEE/ACM international conference on automated software engineering*. 426–437.
- [6] Md Rakibul Islam and Minhaz F Zibran. 2017. Insights into continuous integration build failures. In *2017 IEEE/ACM 14th International Conference on Mining Software Repositories (MSR)*. IEEE, 467–470.
- [7] Xianhao Jin and Francisco Servant. 2022. Which builds are really safe to skip? Maximizing failure observation for build selection in continuous integration. *Journal of Systems and Software* 188 (2022), 111292.
- [8] Mika V Mäntylä, Yuqing Wang, and Jesse Nyyssölä. 2024. Loglead-fast and integrated log loader, enhancer, and anomaly detector. In *2024 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 395–399.
- [9] Mahendra Nath, Jayashree Muralikrishnan, Kuzhanthaiyan Sundararajan, and Madhu Varadarajanna. 2018. Continuous integration, delivery, and deployment: a revolutionary approach in software development. *International Journal of Research and Scientific Innovation (IJRSI)* 5, 7 (2018), 185–190.
- [10] Jesse Nyyssölä and Mika Mäntylä. 2024. Speed and Performance of Parserless and Unsupervised Anomaly Detection Methods on Software Logs. In *2024 IEEE 24th International Conference on Software Quality, Reliability and Security (QRS)*. 657–666. doi:10.1109/QRS62785.2024.00071
- [11] Islem Saidani, Ali Ouni, Moataz Chouchen, and Mohamed Wiem Mkaouer. 2020. Predicting continuous integration build failures using evolutionary search. *Information and Software Technology* 128 (2020), 106392.
- [12] Islem Saidani, Ali Ouni, and Mohamed Wiem Mkaouer. 2022. Improving the prediction of continuous integration build failures using deep learning. *Automated Software Engineering* 29, 1 (2022), 21.
- [13] KK Thyagarajan and G Kalaiarasi. 2021. A review on near-duplicate detection of images using computer vision techniques. *Archives of Computational Methods in Engineering* 28, 3 (2021), 897–916.
- [14] Morteza Zakeri-Nasrabadi, Saeed Parsa, Mohammad Ramezani, Chanchal Roy, and Masoud Ekhtiarzadeh. 2023. A systematic literature review on source code similarity measurement and clone detection: Techniques, applications, and challenges. *Journal of Systems and Software* 204 (2023), 111796.
- [15] Ting Zhang, DongGyun Han, Venkatesh Vinayakara, Ivana Clairine Irsan, Bowen Xu, Ferdian Thung, David Lo, and Lingxiao Jiang. 2023. Duplicate bug report detection: How far are we? *ACM Transactions on Software Engineering and Methodology* 32, 4 (2023), 1–32.